

Unit 6: Backtracking

Concept of Backtracking

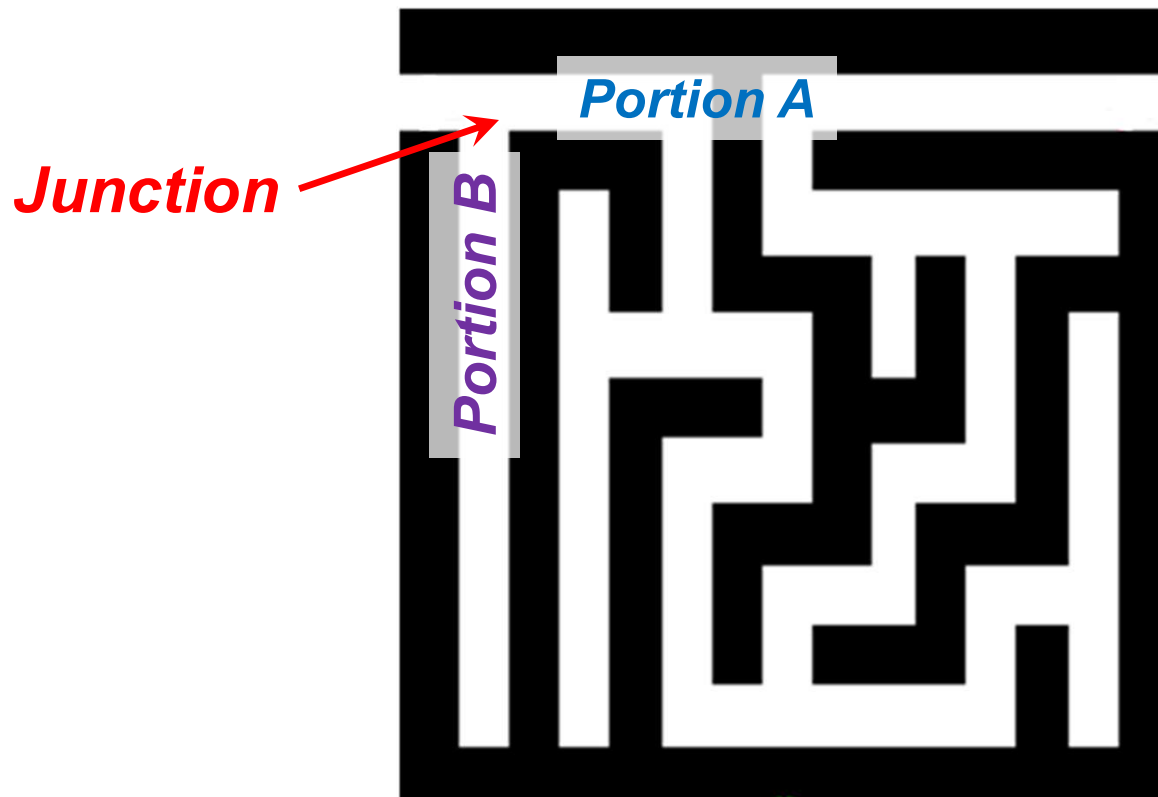
- Backtracking is an algorithmic-technique for solving problems recursively by trying to build a solution incrementally, one piece at a time, removing those solutions that fail to satisfy the constraints of the problem at any point of time
- It is a clever form of exhaustive search technique.
- There are three types of problems in backtracking –
 - Decision Problem – In this, we search for a feasible solution.
 - Optimization Problem – In this, we search for the best solution.
 - Enumeration Problem – In this, we find all feasible solutions.

Concept of Backtracking

- Backtracking is used to solve problems in which a sequence of objects is chosen from a specified set so that the sequence satisfies some criteria.
- Backtracking is a modified depth-first search of a tree.
- Backtracking involves only a tree search.
- Backtracking is the procedure whereby, after determining that a node can lead to nothing but dead nodes, we go back (“backtrack”) to the node’s parent and proceed with the search on the next child.

Backtracking: Idea

- A standard example of backtracking would be going through a maze.
 - At some point, we might have two options of which direction to go:

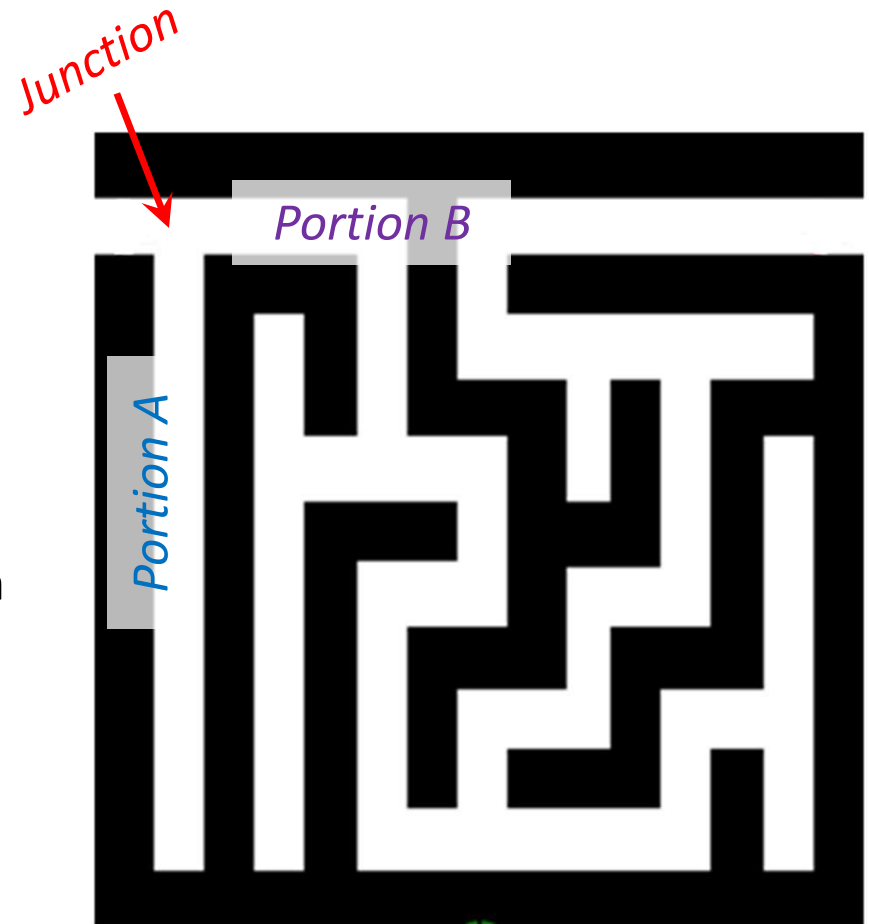


Backtracking

One strategy would be to try going through **Portion A** of the maze.

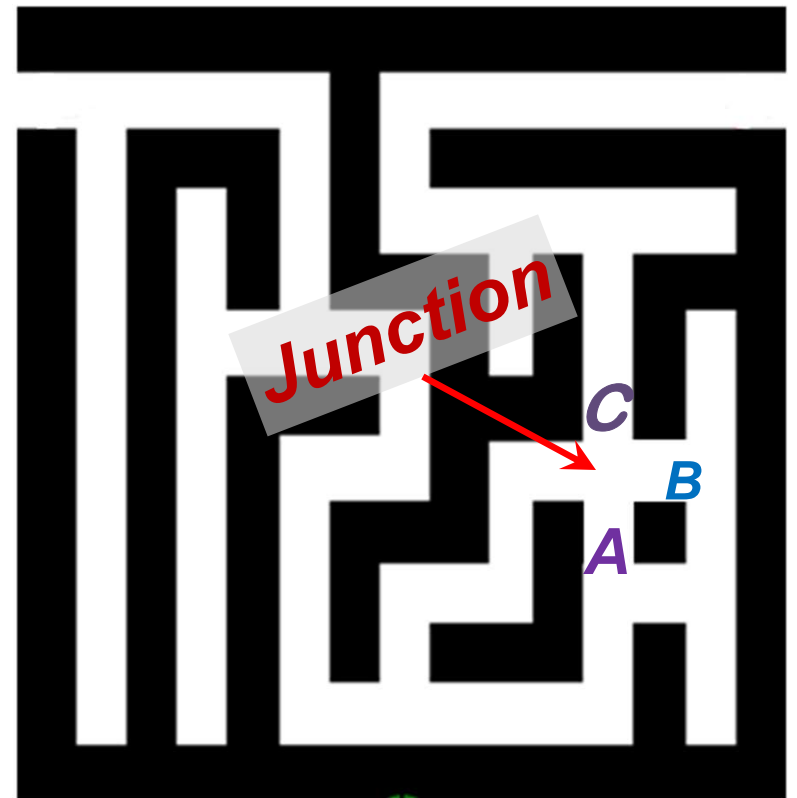
If we get stuck before we find our way out, then we "*backtrack*" to the junction.

At this point in time we know that **Portion A** will *NOT* lead us out of the maze, so we then start searching in **Portion B**

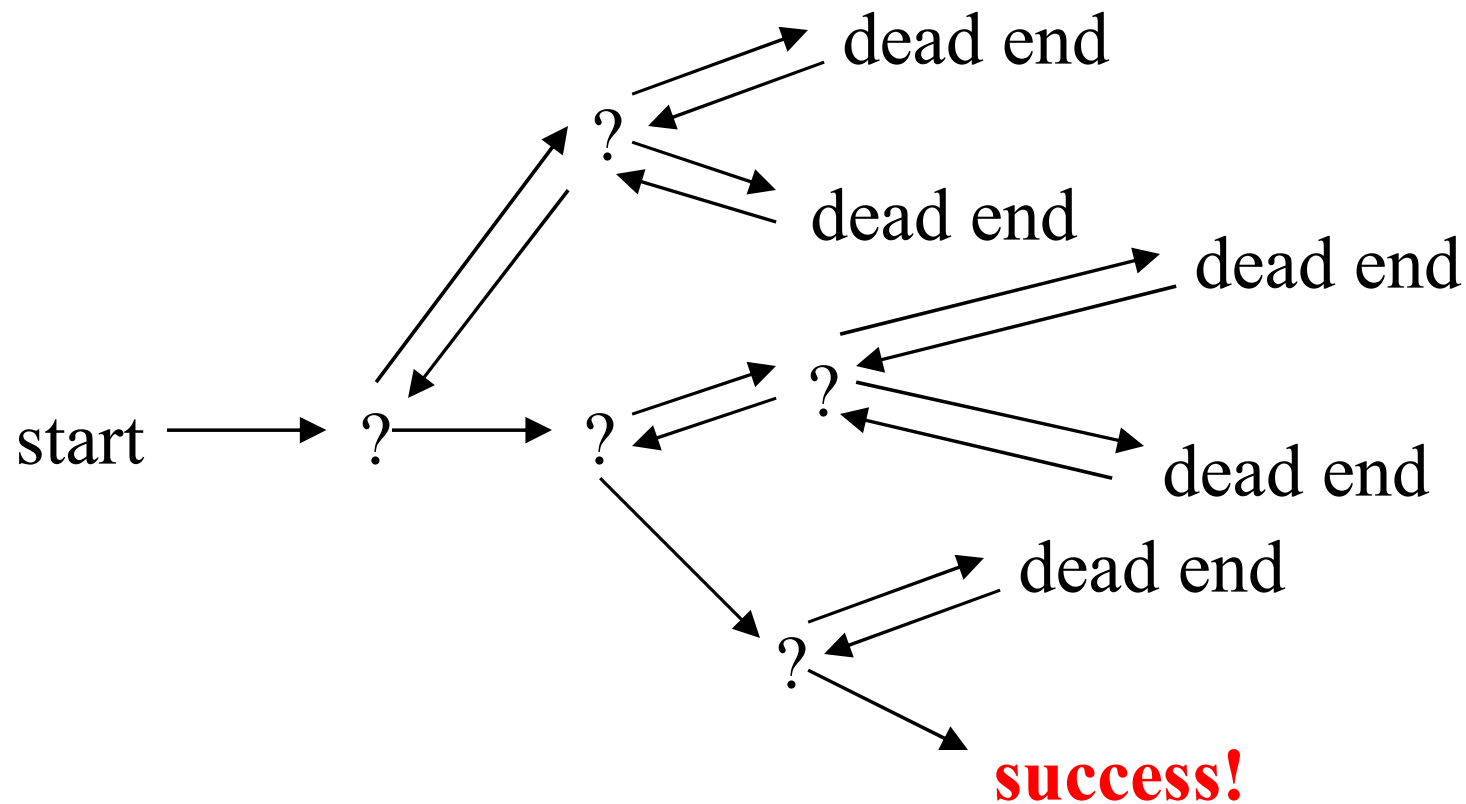


Backtracking

- Clearly, at a single junction we could have even more than 2 choices.
- The backtracking strategy says to try each choice, one after the other,
 - if we ever get stuck, "*backtrack*" to the junction and try the next choice.
- If we try all choices and never found a way out, then there is no solution to the maze.



Backtracking (animation)



Backtracking

- Dealing with the maze:
 - From your start point, you will iterate through each possible starting move.
 - From there, you recursively move forward.
 - If you ever get stuck, the recursion takes you back to where you were, and you try the next possible move.
- Make sure you don't try too many possibilities,
 - Mark which locations in the maze have been visited already so that no location in the maze gets visited twice.
 - If a place has already been visited, there is no point in trying to reach the end of the maze from there again.

Backtracking

The neat thing about coding up backtracking is that it can be done recursively, without having to do all the bookkeeping at once.

- Instead, the stack of recursive calls does most of the bookkeeping
- (i.e., keeps track of which locations we've tried so far.)

Backtracking

- The idea of backtracking is to construct solutions one component at a time and evaluate such partially constructed solutions.
- This partially constructed solution can be developed further without violating the problem constraints.
- It is convenient to implement this kind of processing by constructing a tree of choices being made called the "State Space Tree".
- Its root represents an initial state before the search for the solution begins.
- The nodes of the first level in the tree represent the choices for the first component of a solution and the nodes of a second level represent the choices for the second component and so on.

Promising and Non-Promising Nodes

- A node in the state space tree is promising if it corresponds to the partials constructed solution that may lead to the complete solution otherwise the nodes are called non-promising.
- Leaves of the tree represent either the non-promising dead end or complete solution found by the algorithm.

Recursion Vs Backtracking

- In recursion, the function calls itself until it reaches a base case.
 - Example: Factorial of N, Fibonacci Number etc
- In backtracking, we use recursion to explore all the possibilities until we get the best result for the problem.
 - Example: N-Queen Problem

Subset Sum Problem

- Problem: Given a set of n weights, find a subset of S whose sum is exactly the given value m .
- To solve this problem, let us take an example –
 - $S = \{5, 10, 12, 13, 15, 18\}$, Find subset of S such that sum of subset is 30.
- Here Total weight of Set $S = 73$
- To solve this problem,
 - Start from sum of subset = 0 until we found the sum of subset = $m = 30$
 - Trace the solution tree such as,
 - Go left from the node of the tree that includes the item x_i in subset and right for not including the item x_i .

Subset Sum Problem- Solution

- Example: Solve following problem and draw portion of state space tree $M=30, W = \{5, 10, 12, 13, 15, 18\}$
- Solution:

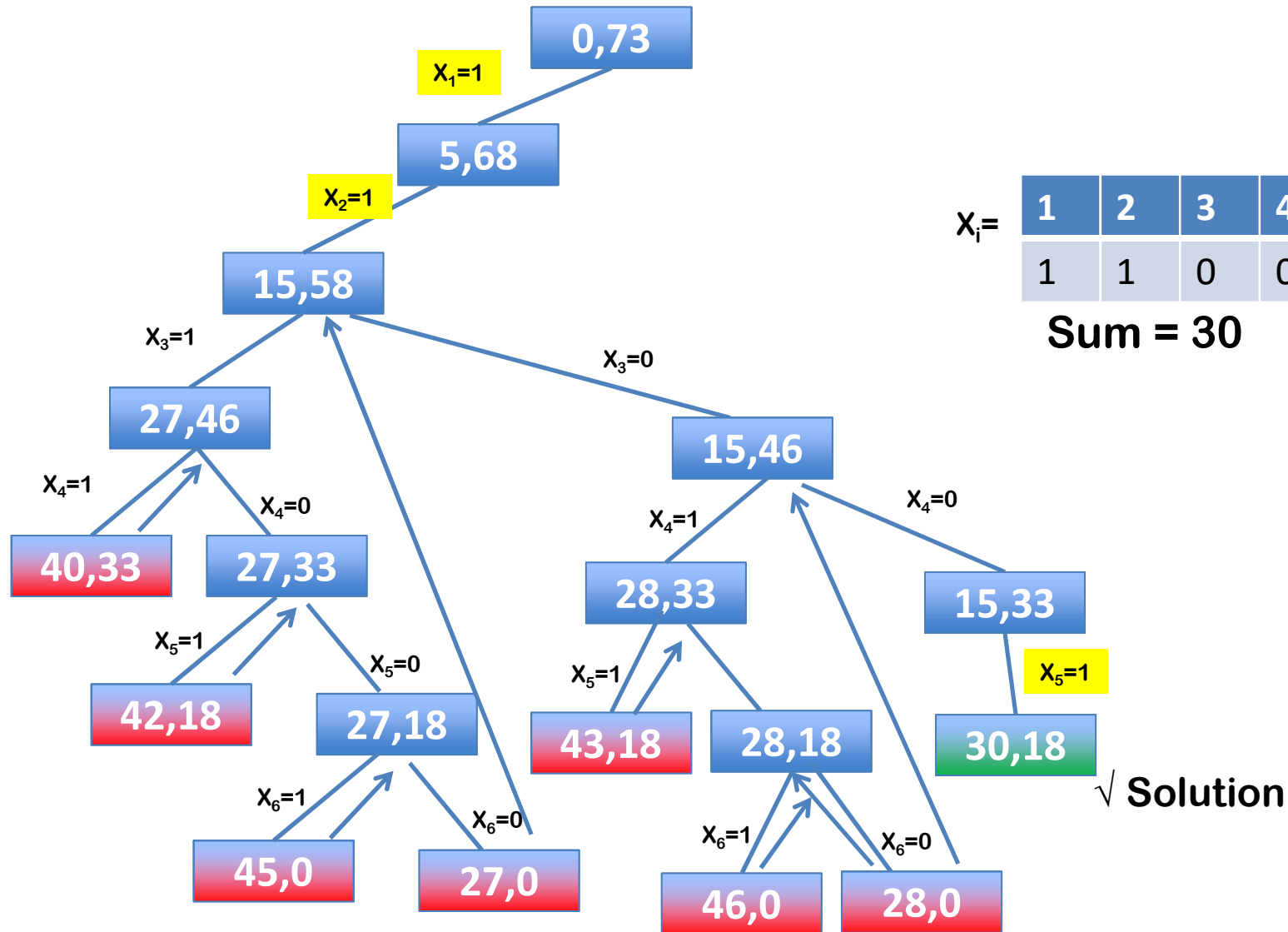
Initially subset = {}	Sum = 0	Description
5	5	Then add next element.
5, 10	15 i.e. $15 < 30$	Add next element.
5, 10, 12	27 i.e. $27 < 30$	Add next element.
5, 10, 12, 13	40 i.e. $40 < 30$	Sum exceeds $M = 30$. Hence backtrack.
5, 10, 12, 15	42	Sum exceeds $M = 30$. Hence backtrack.
5, 10, 12, 18	45	Sum exceeds $M = 30$. Hence backtrack.
5, 12, 13	30	Solution obtained as $M = 30$

Define bounded Function

- Define the Bounded Function as
- If the total weight of object added till now + next object weight $\leq m$,
i.e. $B = \sum_{i=1}^k w_i x_i + w_{k+1} \leq m$ then we can proceed ahead.
- If total object added up to now + weight of remaining objects is $> m$
i.e. $B_1 = \sum_{i=1}^k w_i x_i + \sum_{i=k+1}^n w_i > m$ then we can proceed.

Subset sum: Tree

Start from sum=0, total weight of set =73 and target =30



x_i	1	2	3	4	5	6
	1	1	0	0	1	0

Subset Sum Problem

Backtracking Algorithm for Subset sum Problem

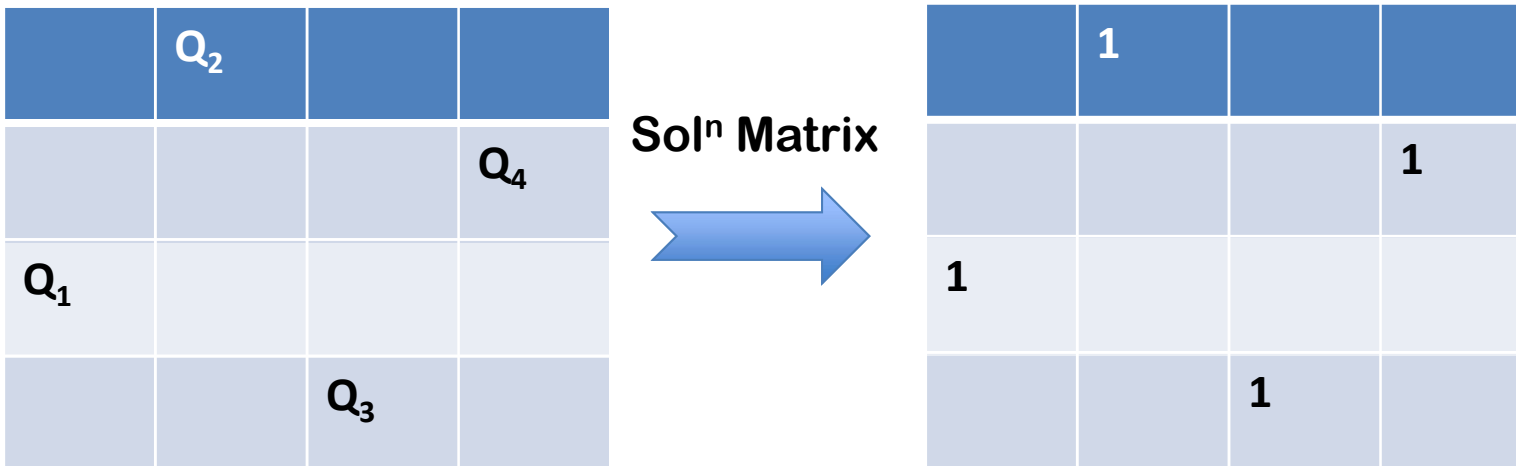
1. Start with empty Set, $\text{sum} = 0$
2. Add the next element from the list to the set
3. If the subset is having sum m , then stop with that subset as solution.
4. If the subset is not feasible or if we have reached the end of the set, then backtrack through the subset until we find the most suitable value.
5. If the subset is feasible ($\text{sum of subset} < m$) then go to step 2
6. If we have visited all the elements without finding a suitable subset and if no backtracking is possible, then stop without solution.

Time Complexity:

Worst case Complexity = $O(2^n)$ since there are 2^n subsets of a set with n numbers to which each set we have to examine for the required sum.

The N-Queen Problem

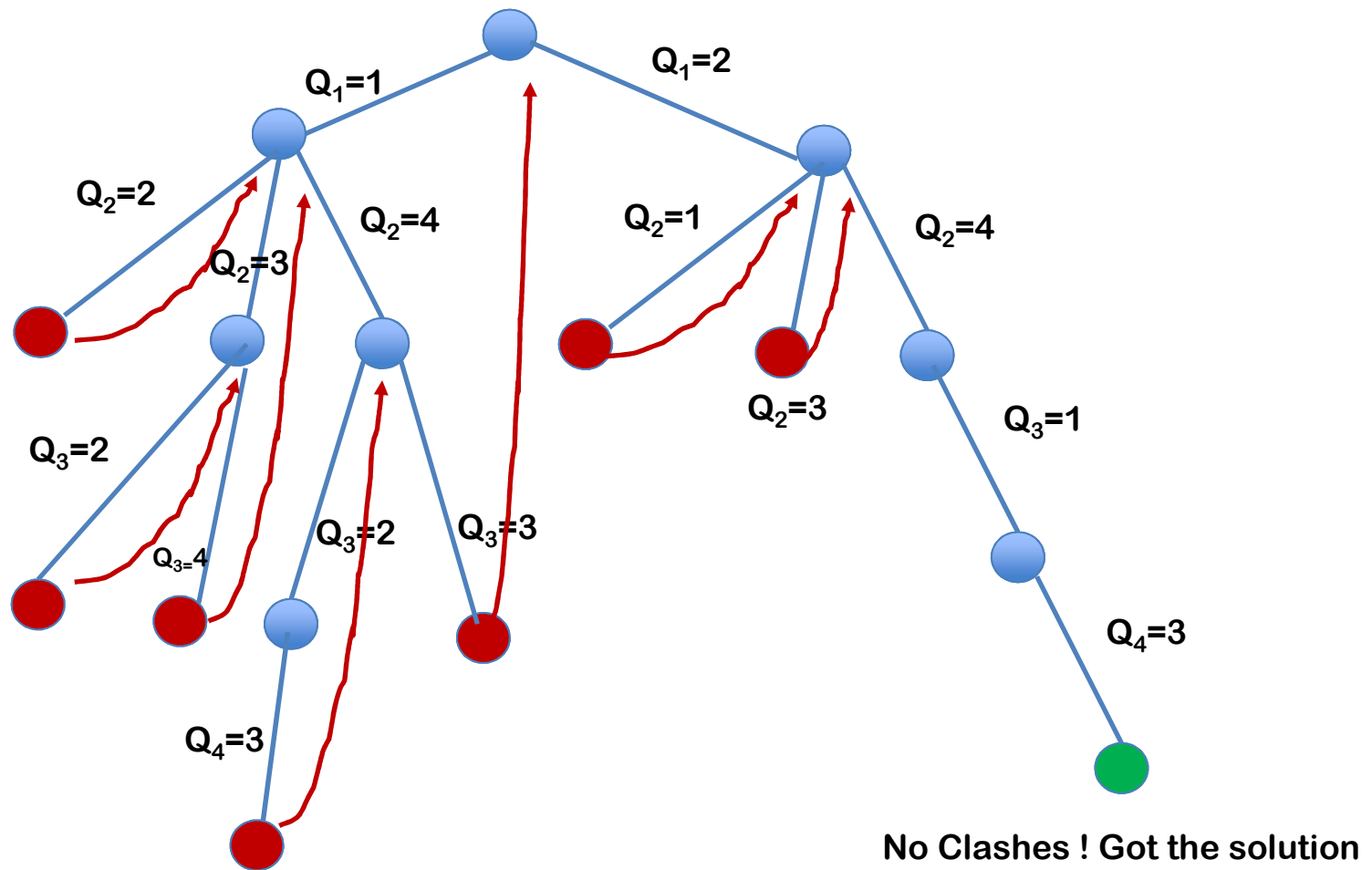
- The N-Queen problem of placing N-Chess queens on a N x N chess board so that no two queen attack each other by being on same row, same column or diagonal.
- If N=1, no solution exists
- If N=2, no solution exists
- If N=3, no solution exists.
- So consider N=4.



The 4-Queen Problem

- There may have more than one solutions.
- For 4-Queen, there are 16 box in chess boards. So there are $16C_4$ combinations to place 4-Queens in 4x4 board.
- For the solution, we can check by making the state space tree.
- Let us construct state-space tree for the solution of 4-Queen problem

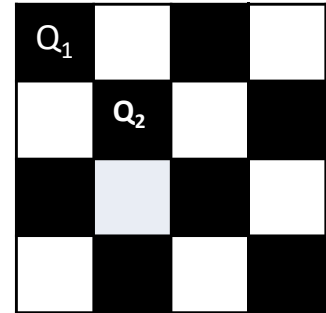
State Space Tree



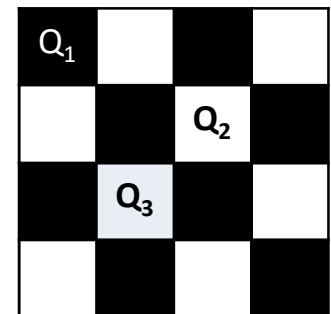
- Here $Q_i=j$ means Queen i in j^{th} Column.

Solving 4-Queen Problem

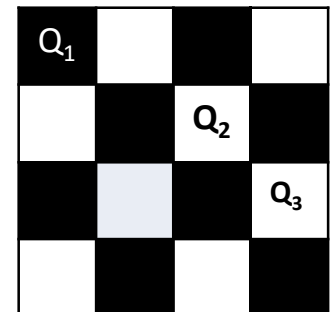
- Start with First Queen in First Column and 2nd Queen in Second Column –Clashes
#So backtrack



- Move Second Queen in Third Column and Place 3rd Queen in Second Column –Clashes
- #So backtrack

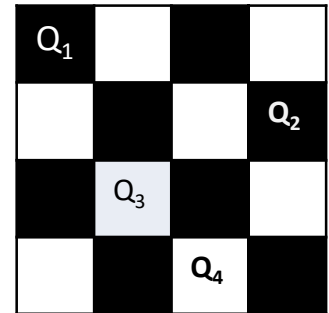


- Place 3rd Queen in 4th Column - Clash
- #So backtrack

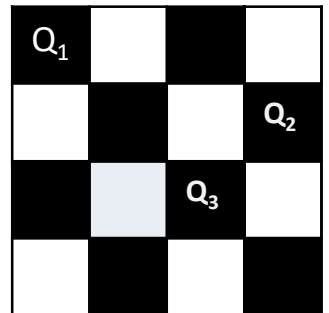


Solving 4-Queen Problem

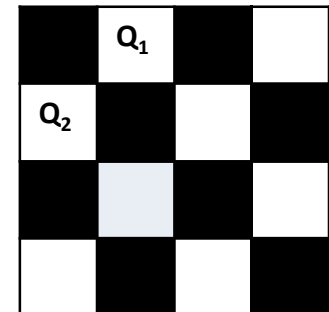
- Q_2 in 4th Column, Q_3 in 2nd Column and Q_4 in 3rd column -Clashes
#So backtrack



- Q_3 in 3rd Column- Clashes
- #So backtrack

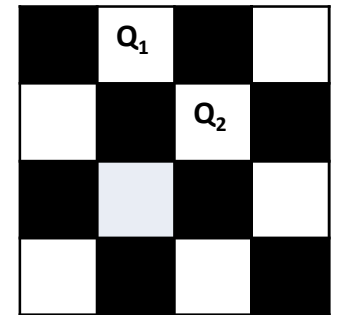


- Place 1st Queen in 2nd Column , Q_2 in 1st Column - Clash
- #So backtrack

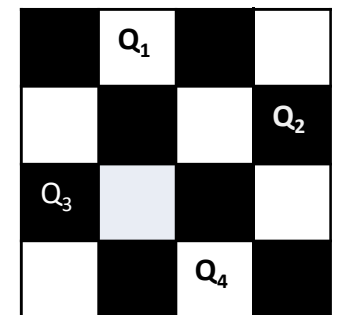


Solving 4-Queen Problem

- Q_2 in 3rd Column - Clashes
#So backtrack



- Q_2 in 4th Column ,
- Q_3 in 1st Column,
- Q_4 in 3rd column , Here no Clashes
So got the solution.



Backtracking Algorithm

- The idea is to place queens one by one in different columns starting from the left most column
- When we place a queen in a column, check for clashes with already placed queens
- In the current column, if there is a row for which there is no clash, mark this row and column as part of the solution.
- If such a row is not found having no clashes, backtrack and return false for that row/column

Backtracking Algorithm

1. Start in the left-most column.
2. If all queens are placed return true
3. Try all row in the current column, Do the following for every tried row.
 - a. If the queen can be placed safely in this row, then mark this [row,col] as part of the solution and recursively check if placing queen here leads to a solution.
 - b. If placing the queen in [row,col] leads to a solution then return true.
 - c. If placing queen doesnot lead to a solution then backtrack this [row,col] and go to step (a) to try another row.
4. If all rows have been tried and nothing worked, return false.

Complexity of N-Queen Problem

- For N Queen,
 - We can select any one of N values for Q_1
 - We can select any one of N-1 values for Q_2
 - We can select any one of N-2 values for Q_3
 -
 - Similarly we can select any one remaining value for Q_N
 - So it tests $N * (N-1) * (N-2) * \dots * 1 = N!$
 - So $T(N) = O(N!)$.

0-1 Knapsack Problem

- Given n items with weight w_i and profit p_i for each item i . The maximum capacity of knapsack is W . The problem is to choose a subset of the items such that
- $\sum_{1 \leq i \leq n} w_i x_i \leq W$ and $\sum_{1 \leq i \leq n} p_i x_i$ is *Maximized*
Where x_i constitute 0 or 1 values i.e. if x_i included in solution it is 1 other wise 0.
- The solution space consists of 2^n distinct ways to assign 0 or 1 values. The time complexity will be exponential
- A good bounding function for this problem is obtained by using upper bound on the value of the best feasible solution obtainable by expanding the live nodes and any of its descendants.
- If this upper bound is not higher than the value of the best solution determined so far then that live node can be killed.

0-1 Knapsack Problem

- This is an optimization problem we can not know if a node contains optimal solution until the search is over.
- We will always expand a promising node to look at its children (may lead to solution), we can not expand a non-promising node.
- **A non-promising node is one if total weight at the node is at least the maximum weight or potential profit for an expansion from that node is not better than the best profit seen so far.**

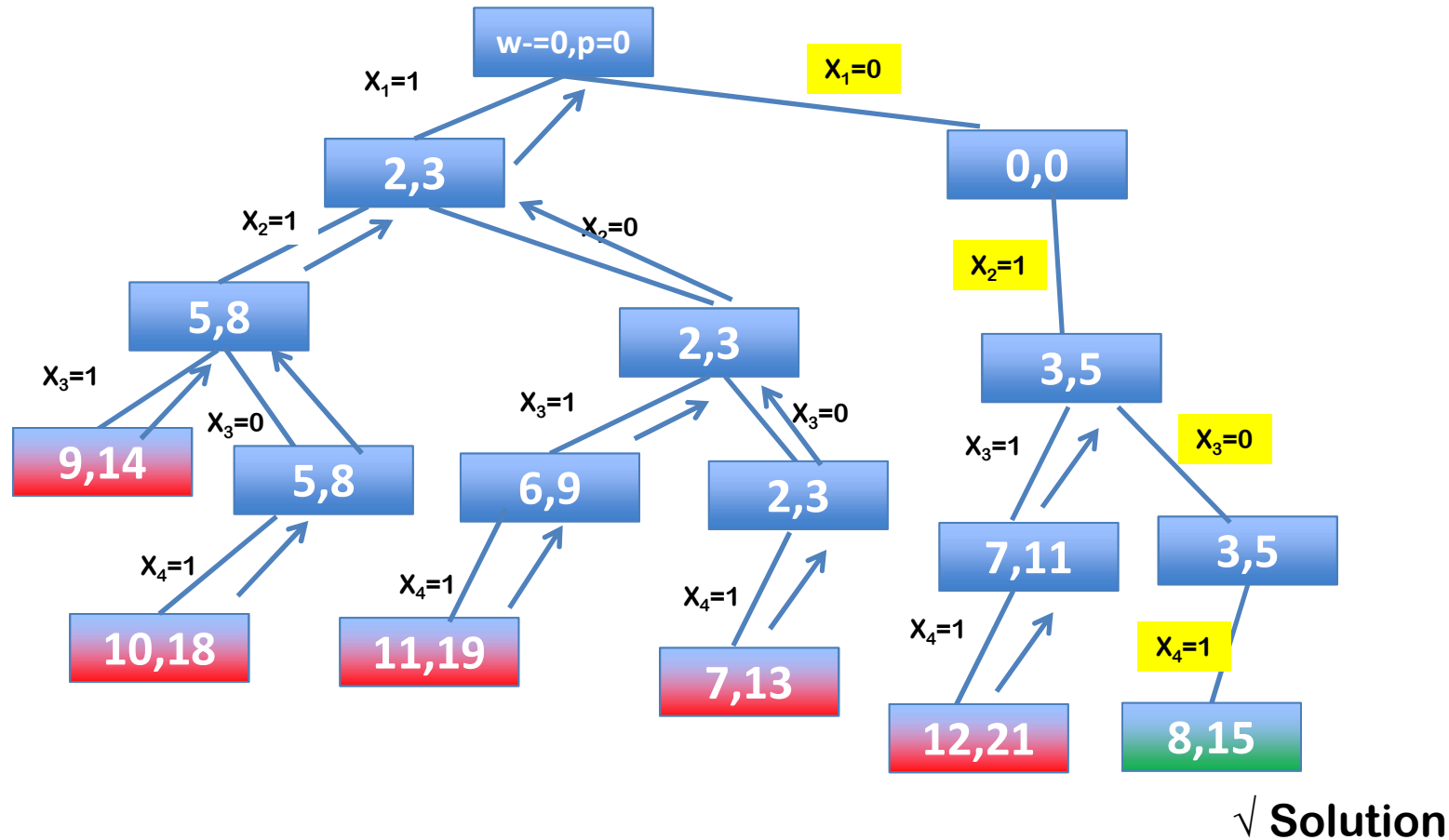
0-1 Knapsack Problem: Example

- Given 4 items with following weight and associated profit each.
- Items-> 1 2 3 4
- W_i -> 2 3 4 5
- P_i -> 3 5 6 10 and Capacity of Knapsack = $W=8$

To find the best solution for this problem, let us construct state-space tree as each node with weight and profit starting with root node with $(w,p) = (0,0)$

State space Tree

Start from root node (0,0)



$x_i =$

1	2	3	4
0	1	0	1